Taller 2: GCC

Etapas de compilación y bibliotecas

José Bernardo Barquero Bonilla
2023150476

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: CE1113 - Sistemas Empotrados

Prof. Dr.-Ing Jeferson González Gómez

20 de Agosto de 2026

Preguntas Teóricas

¿Qué es GCC?

GCC es un conjunto de compiladores que soporta múltiples lenguajes y múltiples arquitecturas de destino, el cual a la hora de ejecutarse invoca una serie de programas independientes que se ejecutan en cadena.

¿Cuáles son las 4 etapas de compilación?

Las cuatro etapas de compilación son las siguientes:

1. Preprocesado.
2. Compilación.
3. Ensamblado.
4. Enlazado.

¿Qué comando debería utilizar para generar el código ensamblador de un archivo fuente, por ejemplo, `calculadora.c`?

El comando a utilizar sería `gcc -S calculadora.c -o calculadora.s`

¿Cuáles es la diferencia entre una biblioteca estática y una dinámica?

Una biblioteca estática cuando un programa se enlaza con ella, el código de las funciones utilizadas se copia dentro del ejecutable final, produciendo binarios grandes y autocontenidos, haciendo así que no dependan de que la biblioteca esté presente en el sistema donde se ejecuten.

Por otro lado, una biblioteca dinámica tiene el código de ella separado del ejecutable, ya que se carga en memoria en tiempo de ejecución, donde el sistema debe poder encontrar el archivo .so en tiempo de ejecución mediante una ruta estándar o en una ruda indicada en una variable de entorno como `LD_LIBRARY_PATH`

Ejercicio 1: Construcción de la biblioteca

A continuación se detallan los comandos utilizados para la construcción de ambas bibliotecas en la Figura 1:

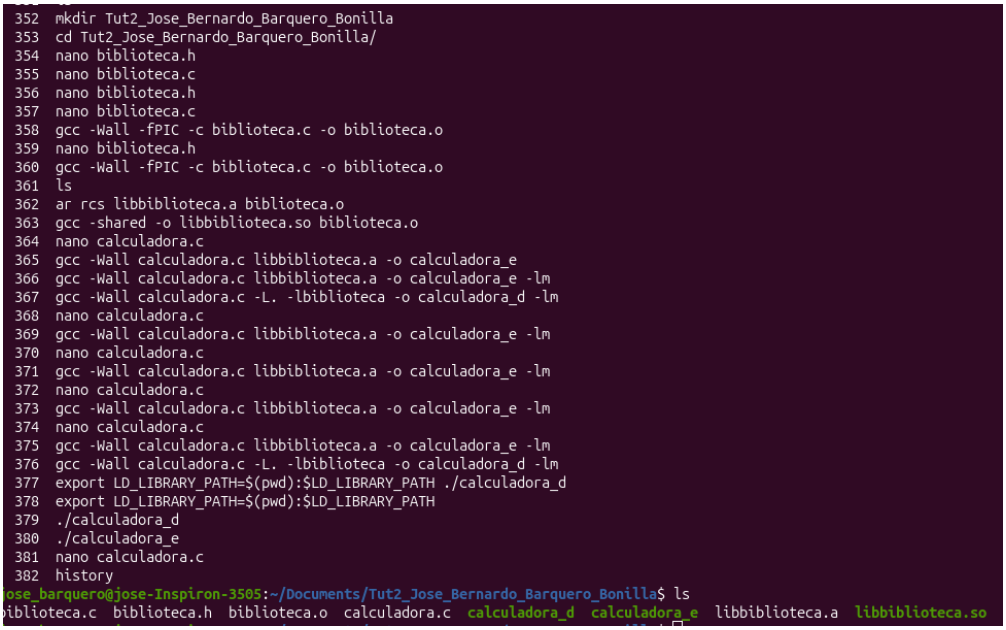


Figure 1: Comandos utilizados para la creación de la biblioteca

Nota: Hay algunos comandos repetidos debido a que tuve unos errores en la compilación debido a errores sintácticos o de comandos mal colocados.

Ejercicio 2: Inspección del binario con herramientas del *tool-chain*

Comparación de tamaño en disco

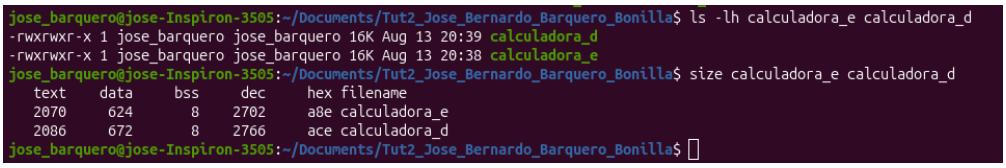


Figure 2: Comandos utilizados para la comparación de tamaños

Note que al comparar con el comando `ls -lh` ambos ejecutables tienen el mismo tamaño debido al tamaño de bloque del sistema de archivos.

Analizando con size, se puede ver una diferencia donde `calculadora_d` es ligeramente mas grande que el otro archivo, esto puede suceder debido a que al ser programas pequeños, el hecho de que se deba realizar de forma dinamica puede generar algun tipo de sobrecarga (ya sea porque se le deba añadir algo al programa para que pueda ser dinamico) al programa al compilarlo de esta forma, haciendo que pese mas. Ademas de que `calculadora_e` solo tiene enlazado estaticamente la biblioteca propia que se realizo, lo cual si este programa usa alguna otra biblioteca estandar (como libc, por ejemplo), hace que siga pesando menos.

Lista de símbolos de las funciones

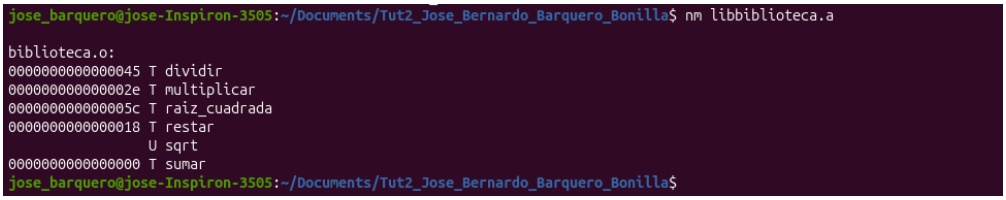


Figure 3: Comando utilizado para ver los símbolos

Note que las funciones de `sumar`, `restar`, `multiplicar` y `dividir` sí están definidas con la letra `T`, esto debido a que se encuentran en la biblioteca realizada, a diferencia de la función `raiz_cuadrada`, la cual se encuentra con la letra `U`, la cual indica que está indefinido, debido a que pertenece a la biblioteca matemática.

Aparición de la biblioteca

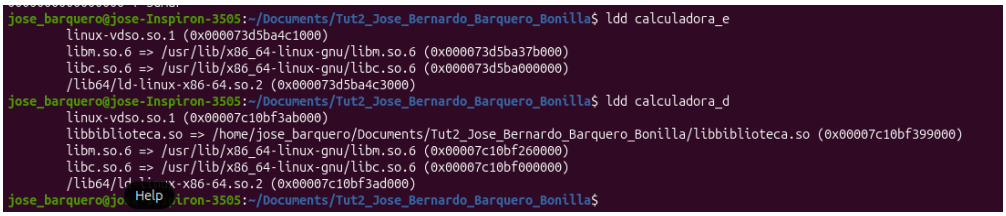


Figure 4: Comando utilizado para ver el uso de la biblioteca

Note que solo aparece la biblioteca realizada para el archivo `calculadora_d` debido a que el enlazamiento realizado fue hecho de forma dinámica, además de añadir el path de la biblioteca realizada, haciendo así que no contenda las instrucciones de lo que se realizó en el interior del archivo principal. En cambio, para `calculadora_e`, este contiene incrustado el código máquina de las funciones implementadas en la biblioteca debido al tipo de enlazamiento que se realizó de forma estática.

Instrucciones de invocación de las funciones

```
jose_barquero@jose-Inspiron-3505:~/Documents/Tut2_Jose_Bernardo_Barquero_Bonilla$ objdump -d calculadora_d

calculadora_d:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    $0x8,%rsp
    1008:       48 8b 05 d9 2f 00 00    mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__>
    100f:       48 85 c0                test   %rax,%rax
    1012:       74 02                   je     1016 <_init+0x16>
    1014:       ff d0                   call   *%rax
    1016:       48 83 c4 08             add    $0x8,%rsp
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <.plt>:
    1020:       ff 35 72 2f 00 00       push   0x2f72(%rip)        # 3f98 <_GLOBAL_OFFSET_TABLE_+0x8>
    1026:       ff 25 74 2f 00 00       jmp    *0x2f74(%rip)        # 3fa0 <_GLOBAL_OFFSET_TABLE_+0x10>
    102c:       0f 1f 40 00             nopl   0x0(%rax)
    1030:       f3 0f 1e fa             endbr64
    1034:       68 00 00 00 00          push   $0x0
    1039:       e9 e2 ff ff ff          jmp    1020 <_init+0x20>
    103e:       66 90                   xchg   %ax,%ax
    1040:       f3 0f 1e fa             endbr64
    1044:       68 01 00 00 00          push   $0x1
```

Figure 5: Comando utilizado para ver el objdump del archivo

```
1216:       89 d6                   mov    %edx,%esi
1218:       89 c7                   mov    %eax,%edi
121a:       e8 91 fe ff ff          call   10b0 <sumar@plt>
121f:       89 c2                   mov    %eax,%edx
1221:       48 8d 05 dc 0d 00 00    lea    0xddc(%rip),%rax        # 2004 <_IO_stdin_used+0x4>
1228:       89 d6                   mov    %edx,%esi
122a:       48 89 c7                mov    %rax,%rdi
```

Figure 6: Identificación de la invocación de la función `sumar`

Resultado usando *strip* y *nm*

```
jose_barquero@jose-Inspiron-3505:~/Documents/Tut2_Jose_Bernardo_Barquero_Bonilla$ nm calculadora_d
0000000000002128 r __abi_tag
0000000000004010 B __bss_start
0000000000004010 b completed.0
                 w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000004000 W data_start
0000000000001130 t deregister_tm_clones
                 U dividir
00000000000011a0 t __do_global_dtors_aux
0000000000003d78 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003d80 d _DYNAMIC
```

Figure 7: Comando utilizado para la versión sin strip

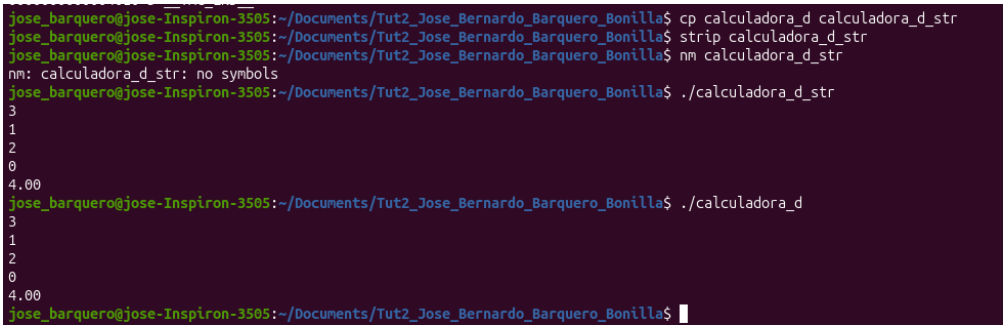


Figure 8: Comandos utilizados para la versión con strip y verificación de su funcionamiento

Note que dio un error al utilizar el comando `strip` en la copia de `calculadora_d_str`, esto debido a que esta copia no contiene ningún tipo de símbolo de debug y tabla de símbolos que permita al comando imprimir algo, lo cual reduce su tamaño, pero como se puede apreciar en la imagen, permite aún ejecutarlo y que su funcionamiento siga siendo el esperado, debido a que eliminar estos símbolos, no afecta a la parte funcional del programa.